

Дополнительные главы дискретной математики:

Алгоритмы и структуры данных

Лекция 7. Сортировка слиянием. Быстрая сортировка.

Ключарёв Пётр Георгиевич

pgkl@yandex.ru

twitter.com/klyucharev

Кафедра ИУ8 «Информационная безопасность»
МГТУ им. Н.Э. Баумана

2013 г.

Разделяй и властвуй

Три основных этапа

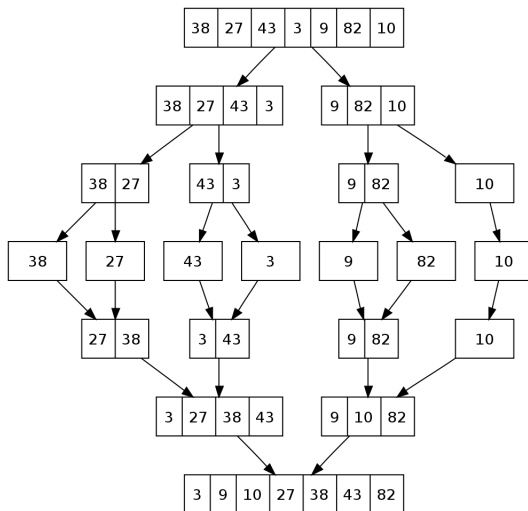
- 1 Разделить задачу на подзадачи
- 2 Решить подзадачи рекурсивно
- 3 Если размер задачи достаточно мал — решить методом грубой силы.

Сортировка слиянием

Алгоритм 1 Сортировка слиянием. Сортирует подмассив $A[p \dots r]$

```
1: procedure MERGESORT( $A, p, r$ )
2:   if  $p < r$  then
3:      $q := \lfloor (p + r) / 2 \rfloor$ 
4:     MERGESORT( $A, p, q$ )
5:     MERGESORT( $A, q + 1, r$ )
6:     MERGE( $A, p, q, r$ )
```

Пример работы алгоритма сортировки слиянием



Сортировка слиянием

Алгоритм 2 Слияние $A[p \dots q]$ и $A[q + 1 \dots r]$

```
1: procedure MERGE( $A, p, q, r$ )
2:    $n_1 := q - p + 1$ 
3:    $n_2 := r - q$ 
4:   Создать массивы  $L[1 \dots n_1 + 1]$  и  $R[1 \dots n_2 + 1]$ .
5:   for  $i := 1$  to  $n_1$  do  $L[i] := A[p + i - 1]$ 
6:   for  $j := 1$  to  $n_2$  do  $R[j] := A[q + j]$ 
7:    $L[n_1 + 1] := \infty$ ;  $R[n_2 + 1] := \infty$ 
8:    $i := 1$ ;  $j := 1$ 
9:   for  $k := p$  to  $r$  do
10:    if  $L[i] \leq R[j]$  then
11:       $A[k] := L[i]$ 
12:       $i := i + 1$ 
13:    else
14:       $A[k] := R[j]$ 
15:       $j := j + 1$ 
```

Сложность

Сложность слияния: $\Theta(n)$. Сложность сортировки слиянием:

$$T(n) = \begin{cases} \Theta(1) & \text{если } n = 1, \\ 2T(n/2) + \Theta(n) & \text{если } n > 1. \end{cases}$$

$$T(n) = O(n \log n)$$

Быстрая сортировка

Алгоритм 3 Быстрая сортировка. Сортирует подмассив $A[l + 1] \dots A[r]$.

```
1: procedure QUICKSORT( $A, l, r$ )
2:   if  $l \neq r$  then
3:     Выбрать случайное число  $s \in \{l + 1, \dots, r\}$ 
4:      $b := A[s]$ 
5:      $(ll, rr) := \text{PARTITION}(A, l, r, b)$ 
6:     QUICKSORT( $l, ll$ )
7:     QUICKSORT( $rr, r$ )
```

Процедура PARTITION

Алгоритм 4 Переставляет элементы подмассива $A[l+1] \dots A[r]$ так, чтобы сначала стояли элементы меньшие b , затем равные b , а затем большие b .

```
1: procedure PARTITION( $A, l, r, b$ )
2:    $m := l$ 
3:   while  $m \neq r$  do
4:     if  $A[m+1] = b$  then  $m := m + 1$ 
5:     else if  $A[m+1] > b$  then
6:       Обменять  $A[m+1]$  и  $A[r]$ 
7:        $r := r - 1$ 
8:     else
9:       Обменять  $A[m+1]$  и  $A[l+1]$ 
10:     $l := l + 1$ 
11:     $m := m + 1$ 
12:  return  $(l, r)$ 
```

Сложность алгоритма: $\Theta(n)$.

Сложность быстрой сортировки

Время работы в худшем случае: $O(n^2)$

Время работы в среднем.

(1) Будем считать, что сначала каждому элементу случайно присваиваются различные ранги (от 1 до n), а на каждом шаге выбирается элемент минимального ранга.

(2) Каждый элементу сопоставим его номер в отсортированном массиве

(3) Для каждых двух номеров i, j из $\{1, \dots, n\}$ через $p(i, j)$ обозначим вероятность того, что элементы с этими номерами будут сравниваться друг с другом. Например, $p(i, i + 1) = 1$, поскольку соседние по номеру элементы должны быть сравнены обязательно (сравнения с другими элементами их не различают).

Сложность быстрой сортировки (продолжение)

(4) Заметим, что $p(i, i + 2) = 2/3$. В самом деле, сравнения не произойдёт в том и только том случае, когда из трёх элементов с номерами i , $i + 1$ и $i + 2$ элемент $i + 1$ имеет наименьший ранг. Аналогично, $p(i, i + k) = 2/(k + 1)$ (элементы с номерами i и $i + k$ сравниваются, если среди $k + 1$ элементов $i, i + 1, \dots, i + k$ один из двух крайних имеет наименьший ранг).

(5) Мат. ожидание числа сравнений можно разбить в сумму $\sum m(i, j)$ мат. ожиданий числа сравнений между элементами с номерами i и j . Но поскольку два данных элемента сравниваются не более одного раза, $m(i, j) = p(i, j)$. Таким образом, получаем выражение для мат. ожидания числа сравнений:

$$\sum_{1 \leq i < j \leq n} \frac{2}{j - i + 1}.$$

Сложность быстрой сортировки (окончание)

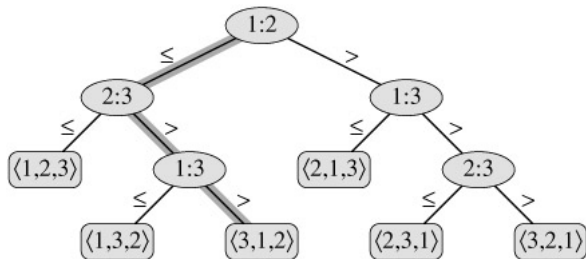
$$\sum_{1 \leq i < j \leq n} \frac{2}{j - i + 1}.$$

(6) Группируя в этой сумме равные члены и учитывая, что $1 + 1/2 + 1/3 + \dots + 1/k = O(\log k)$, получаем оценку

$$O(n \log n)$$

для математического ожидания времени работы быстрой сортировки.

Сортировка и решающие деревья



Количество листьев: $\geq n!$.

Формула Стирлинга

Имеет место формула Стирлинга:

$$n! \sim \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$$

Докажем её усеченную версию.

Теорема

$$n! \geq (n/e)^n$$

Доказательство.

$$\ln n! = \sum_{k=1}^n \ln k \geq \int_1^n \ln x dx = n \ln n - n + 1$$

Следовательно,

$$n! \geq e \frac{n^n}{e^n} \geq \left(\frac{n}{e}\right)^n$$



Нижняя оценка сложности задачи сортировки в худшем случае

Теорема

Высота любого решающего дерева, сортирующего n элементов, есть $\Omega(n \log n)$

Доказательство.

Среди листьев сортирующего дерева должны быть представлены все перестановки n элементов, число листьев не менее $n!$.

Двоичное дерево высоты h имеет не более 2^h листьев, имеем $n! \leq 2^h$.

Учитывая, что $n! \geq (n/e)^n$, получаем $(n/e)^n \leq 2^h$.

Логарифмируя, получаем: $h \geq n \log_2 n - n \log_2 e = \Omega(n \log n)$. □

Следствие. Пирамидальная сортировка и сортировка слиянием асимптотически оптимальны.

Упражнения

- 1 Почти любой алгоритм можно немного изменить, радикально уменьшив время его работы в лучшем случае. Как?
- 2 Дана последовательность чисел $x_1, x_2, \dots, x_n$. Покажите, что за время $\Theta(n \log n)$ можно определить, есть ли в этой последовательности два одинаковых числа.
- 3 показать работу сортировки слиянием для массива $A = \langle 3, 41, 52, 26, 38, 57, 9, 49 \rangle$.
- 4 Поиск количества различных чисел в данном массиве
- 5 Дан массив S из n действительных чисел, а также число x . Как за время $\Theta(n \log n)$ определить, можно ли представить x в виде суммы двух элементов массива S ?

И снова упражнения

- 1 Профессор разработал компьютер, который поддерживает «тройные ветвления»: после одного-единственного сравнения $a_i : a_j$ управление может быть передано в одно из трёх мест программы, в зависимости от того, какое из соотношений выполнено: $a_i < a_j$, $a_i = a_j$ или $a_i > a_j$. Он надеется, что благодаря таким сравнениям сортировку n элементов можно провести асимптотически быстрее, чем за время $\Omega(n \log n)$. Прав ли профессор?
- 2 Найти алгоритм минимальной сложности для нахождения самого тяжелого камня, если есть n камней и простые весы.
- 3 Дано $2n$ камней. Найти алгоритм нахождения самого легкого и самого тяжелого камня, требующий $3n - 2$ взвешиваний.